

R intermédiaire

Atelier intensif

Guillaume Boucher

INSPQ

Le 10 septembre 2026

R Intermédiaire

Contrôle du flux

- Conditions
- Fonctions
- Boucles

Conditions

Utilité des conditions

Une condition permet à R de prendre une décision selon qu'une règle est vraie (TRUE) ou fausse (FALSE).

Exemple :

Si l'âge est de 65 ans ou plus,
alors la personne est considéré comme un aîné.
Sinon,
elle n'est pas considéré comme un aîné.

```
age <- 72
if (age >= 65) {
  print("Aîné")
} else {
  print("Non aîné")
}
```

```
#> [1] "Aîné"
```

Structure d'une condition

```
if (condition) {  
    action si TRUE  
} else {  
    action si FALSE  
}
```

Plusieurs conditions

Lorsqu'il y a plusieurs conditions, la deuxième condition et les suivantes se font avec `else if`.

```
age <- 45
if (age >= 65) {
  print("Aîné")
} else if (age >= 18) {
  print("Adulte")
} else if (age < 18) {
  print("Mineur")
} else {
  print(NA)
}
```

```
#> [1] "Adulte"
```

Conditions dans une condition

Les conditions peuvent s'imbriquer les unes dans les autres

```
if (note >= 60) {  
    if (note >= 90) {  
        print("A")  
    } else {  
        print("Réussite")  
    }  
} else {  
    if (note >= 50) {  
        print("Échec de peu")  
    } else {  
        print("Échec")  
    }  
}
```

Condition sur un vecteur

`if` est utilisé pour prendre une décision dans un programme.
On utilise `ifelse()` pour appliquer une règle à toutes les valeurs d'un vecteur ou d'une colonne.

```
age <- c(72, 54, 22, 65, 18)
ifelse(test = age >= 65, yes = "Aîné", no = "Adulte")
```

```
#> [1] "Aîné"    "Adulte" "Adulte" "Aîné"    "Adulte"
```

```
age <- c(72, 54, 22, 65, 7)
ifelse(
  age >= 65,
  "Aîné",
  ifelse(age > 18,
    "Adulte",
    "Mineur")
)
```

```
#> [1] "Aîné"    "Adulte" "Adulte" "Aîné"    "Mineur"
```


Fonctions

Problème ?

Voyez-vous un problème ici ?

```
age <- 88
if (age >= 65) {
  print("Aîné")
} else {
  print("Non aîné")
}
```

```
age <- 44
if (age >= 65) {
  print("Aîné")
} else {
  print("Non aîné")
}
```

```
age <- 46
if (age >= 65) {
  print("Aîné")
} else {
  print("Non aîné")
}
```

```
age <- 36
if (age >= 65) {
  print("Aîné")
} else {
  print("Non aîné")
}
```

```
age <- 17
if (age >= 65) {
  print("Aîné")
} else {
  print("Non aîné")
}
```

```
age <- 5
if (age >= 65) {
  print("Aîné")
} else {
  print("Non aîné")
}
```

Utilité d'une fonction

Une fonction permet d'enregistrer une série d'instructions sous un nom afin de pouvoir les réutiliser.

```
groupe_age <- function(age) {  
  if (age >= 65) {  
    return("Aîné")  
  } else {  
    return("Non aîné")  
  }  
}
```

```
class(groupe_age)
```

```
#> [1] "function"
```

```
groupe_age(88)
```

```
#> [1] "Aîné"
```

```
groupe_age(36)
```

```
#> [1] "Non aîné"
```

```
grp_age <- groupe_age(44)  
grp_age
```

```
#> [1] "Non aîné"
```

Structure d'une fonction

Nom de la fonction

On commence par inscrire le nom de la fonction qu'on veut créer :
`groupe_age`.

Créer la fonction

On crée un objet fonction de la même manière qu'un `vector`, une `list`, un `data.frame`...

```
groupe_age <- function() {...}
```

Arguments

On inscrit les arguments à l'intérieur des parenthèses.

```
groupe_age <- function(argument1, argument2)
```

Une fonction peut ne pas avoir d'argument.

```
msg_bienvenue <- function() {  
  paste("Aujourd'hui nous sommes le", Sys.Date())  
}  
msg_bienvenue()
```

```
#> [1] "Aujourd'hui nous sommes le 2026-09-04"
```

Prenons par exemple la fonction `median` :

```
args(median)
```

```
#> function (x, na.rm = FALSE, ...)  
#> NULL
```

- **x** : *an object for which a method has been defined, or a numeric vector containing the values whose median is to be computed.*
- **na.rm = FALSE** : *a logical value indicating whether NA values should be stripped before the computation proceeds.*
- **...** : *potentially further arguments for methods; not used in the default method.*

Le premier argument `x` n'a pas de valeur par défaut, il est donc **nécessaire** sinon il y aura une erreur.

```
median()
```

```
#> Error in `median.default()`:  
#> ! l'argument "x" est manquant, avec aucune valeur par défaut
```

L'argument `na.rm` a la valeur `FALSE` par défaut alors il n'est pas nécessaire de l'inclure si la valeur par défaut est celle qu'on veut utiliser.

```
median(x = 1:4)
```

```
#> [1] 2.5
```

```
median(x = 1:4, na.rm = FALSE)
```

```
#> [1] 2.5
```

Il n'est pas nécessaire de nommer les arguments lorsque l'ordre est respecté.

```
median(1:4, FALSE)
```

```
#> [1] 2.5
```

Au contraire, il est nécessaire de nommer les arguments lorsque l'ordre n'est pas respecté.

```
median(na.rm = FALSE, x = 1:4)
```

```
#> [1] 2.5
```


Corps de la fonction

Exemple où on veut calculer l'indice de masse corporelle (IMC) :

```
poids <- 80 # kg  
taille <- 1.75 # mètres  
poids / taille ^ 2
```

```
#> [1] 26.12245
```

Les variables `poids` et `taille` sont les arguments de la fonction

```
imc <- function(poids, taille) {...}
```

et le calcul de la variable `imc_val` est le corps de la fonction, les instructions à exécuter.

```
imc <- function(poids, taille) {  
  poids / taille ^ 2  
}  
imc(poids = 80, taille = 1.75)
```

Retour d'une fonction

R accepte deux styles pour retourner la valeur d'une fonction :

1. Retour implicite : la dernière ligne de code.

```
imc <- function(poids, taille) {  
  poids / taille ^ 2  
}
```

1. Retour explicite : utiliser `return()`.

```
imc <- function(poids, taille) {  
  return(poids / taille ^ 2)  
}  
  
imc <- function(poids, taille) {  
  imc_val <- poids / taille ^ 2  
  return(imc_val)  
}
```

Boucles

Pourquoi utiliser une boucle

Une boucle permet de répéter un même bloc de code plusieurs fois de suite.

Analogie simple :

- Sans boucle, on dit : *Bonjour A, bonjour B, bonjour C, ..., bonjour Z.*
- Avec une boucle, on dit : *Pour chaque personne dans la liste, dire bonjour.*

For

S'utilise quand on sait exactement combien de fois on veut répéter l'action.

```
for (i in 1:3) {  
  print(i)  
}
```

```
#> [1] 1  
#> [1] 2  
#> [1] 3
```

```
for (lettre in c("a","b","c")) {  
  print(lettre)  
}
```

```
#> [1] "a"  
#> [1] "b"  
#> [1] "c"
```

Reviens à exécuter manuellement :

```
i <- 1  
print(i)  
i <- 2  
print(i)  
i <- 3  
print(i)
```

```
i <- "a"  
print(i)  
i <- "b"  
print(i)  
i <- "c"  
print(i)
```

While

Utile quand on ne sait pas combien de tours il faut faire, mais on sait **quand** on doit arrêter.

```
while (x <= 5) {  
  print(x)      # 2, 3, 4, 5  
  x <- x + 1    # 3, 4, 5, 6 est plus grand que 5 -> fin boucle  
}
```

```
#> [1] 2  
#> [1] 3  
#> [1] 4  
#> [1] 5
```

lapply

C'est une fonction en R qui répète une même action sur chaque élément et renvoie le résultat sous la forme d'une liste.

```
args(lapply)
```

```
#> function (X, FUN, ...)  
#> NULL
```

- X : a vector (atomic or list) or an expression object. Other objects (including classed objects) will be coerced by `base::as.list`.
- FUN : the function to be applied to each element of X: see 'Details'. In the case of functions like `+`, `%*%`, the function name must be backquoted or quoted.

Exemple disponible dans la documentation :

```
x <- list(a = 1:10, beta = exp(-3:3),  
          logic = c(TRUE, FALSE, FALSE, TRUE))  
lapply(x, mean)
```

```
#> $a  
#> [1] 5.5  
#>  
#> $beta  
#> [1] 4.535125  
#>  
#> $logic  
#> [1] 0.5
```


Exemple avec une fonction personnalisée :

```
# Prendre note que 1:5 est un vecteur comparativement à l'exemple  
# précédent qui était une liste.  
lapply(1:5, function(x) {  
   $x^2 + 4$   
})
```

```
#> [[1]]  
#> [1] 5  
#>  
#> [[2]]  
#> [1] 8  
#>  
#> [[3]]  
#> [1] 13  
#>  
#> [[4]]  
#> [1] 20  
#>  
#> [[5]]  
#> [1] 29
```

Exemple sur un `data.frame` :

```
lapply(iris[, 1:3], mean)
```

```
#> $Sepal.Length  
#> [1] 5.843333  
#>  
#> $Sepal.Width  
#> [1] 3.057333  
#>  
#> $Petal.Length  
#> [1] 3.758
```

Comparativement à une boucle `for` :

```
for (col in names(iris[, 1:3])) {  
  print(mean(iris[[col]]))  
}
```

```
#> [1] 5.843333  
#> [1] 3.057333  
#> [1] 3.758
```

sapply

Le l dans lapply est pour list alors que le s dans sapply est pour simplifié

```
sapply(iris[, 1:3], mean)
```

```
#> Sepal.Length Sepal.Width Petal.Length  
#>      5.843333      3.057333      3.758000
```

```
sapply(iris[, 1:3], mean, simplify = FALSE) # Équivalent à lapply
```

```
#> $Sepal.Length  
#> [1] 5.843333  
#>  
#> $Sepal.Width  
#> [1] 3.057333  
#>  
#> $Petal.Length  
#> [1] 3.758
```

À retenir

- Les **conditions** permettent de prendre des décisions.
- Les **fonctions** permettent de réutiliser du code.
- Les **boucles** permettent de répéter un traitement.
- Ensemble, elles permettent d'écrire du code plus simple, robuste et maintenable.

Question à se poser :

Est-ce que je suis en train de répéter quelque chose qui pourrait être automatisé ?